

Quick questions



Why would an airplane crash if it ran your laptop OS?



Why doesn't your coffee machine run Windows, but your PC does?



Why can't your fridge send emails using Windows?



Different devices and applications require operating systems tailored for speed, reliability, and hardware constraints.

Operating-System Structure

—

By : Bilal Ahmed

Learning Outcomes

1

Identify different categories of operating systems based on their usage and environment

2

Differentiate between general-purpose and specialized operating systems

3

Explain where specific OS categories are used with real-world examples

4

Select an appropriate operating system type for a given application or device



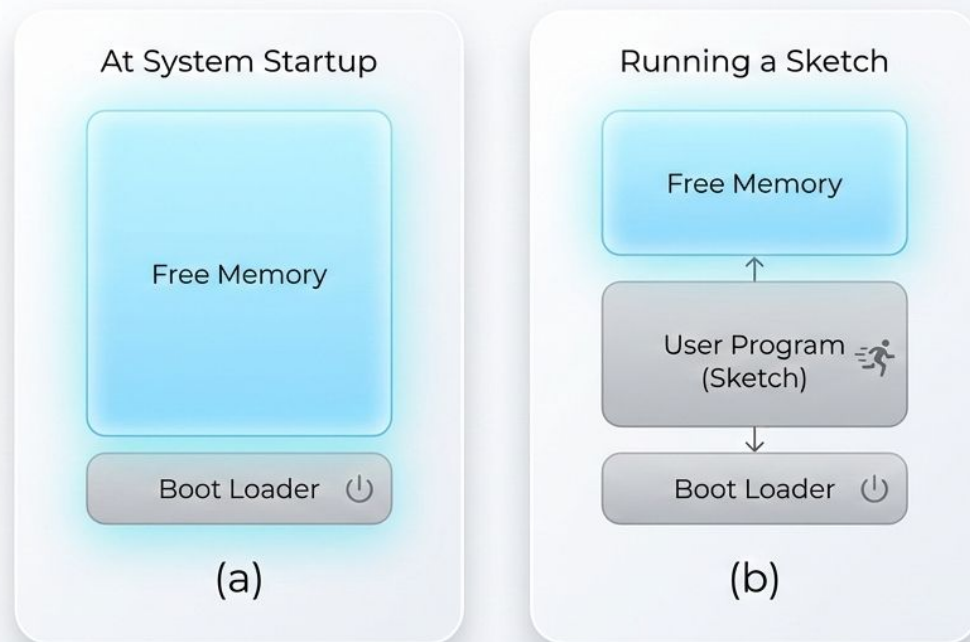
Single-Tasking vs Multitasking Operating System

- Single-Tasking OS:** Executes only one program at a time
 - Multitasking OS:** Executes multiple programs by rapid CPU switching
 - Working Difference:** Single-tasking has no context switching; multitasking uses time slicing
 - Examples:** Single-tasking → Arduino, MS-DOS | Multitasking → Windows, Linux
-

Arduino: Single-Tasking Hardware Platform

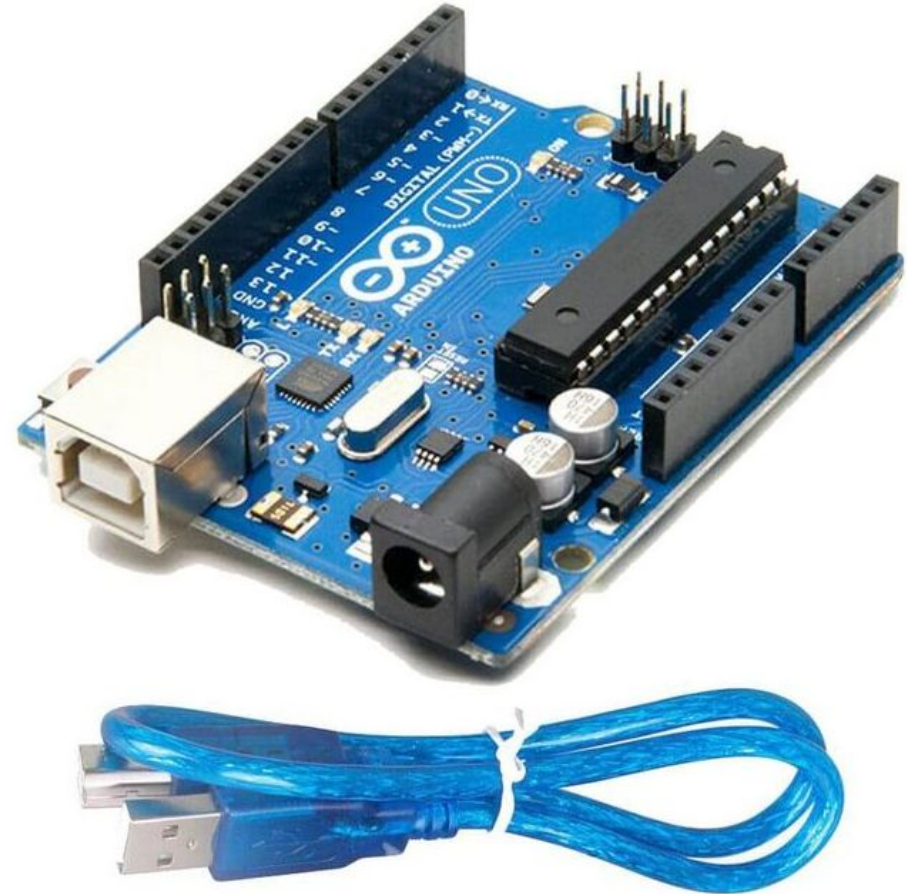
- Hardware:** Microcontroller + input sensors (light, temperature, pressure, etc.)
- Programming:** Write sketch on PC → Upload via USB to Arduino flash memory
- Bootloader:** Small software loads the sketch into memory
- Execution:** Sketch runs directly, waiting for programmed events
- Example:** Temperature sensor triggers fan motor
- Single-tasking:** Only one sketch runs at a time; new sketch replaces old one
- User Interface:** No OS-based UI; interacts only via sensors/outputs

Arduino Memory Execution



Arduino: Single-Tasking Hardware Platform

- Arduino **does not have a full OS**
- Single-tasking embedded system
- Lightweight, specialized for direct hardware control
- Cannot run multitasking or user-level applications like PC OS



FreeBSD Operating System



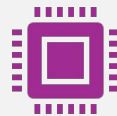
Unix-based, open-source OS derived from Berkeley UNIX (BSD)



First released in **1993**



Supports **multitasking** and **multi-user** operation



Used in **servers and networking systems** (e.g., Netflix, PlayStation)

Multitasking in FreeBSD

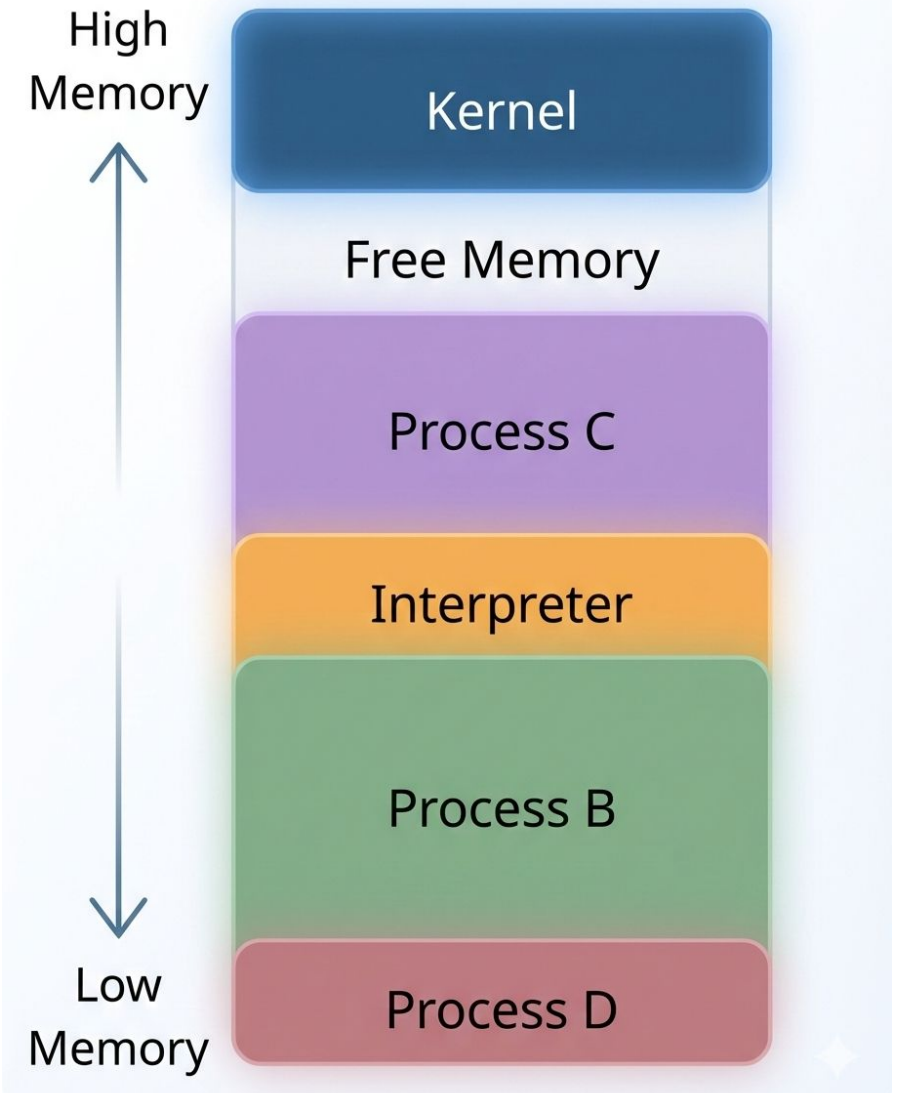
FreeBSD allows multiple programs to run at the same time

The shell accepts commands while other programs execute

Programs can run in the background without blocking the shell

Each program finishes by returning a success or error status

Example: **Netflix streaming servers**



Batch Operating System

Executes jobs in batches with no user interaction

Jobs are collected, queued, and processed sequentially

CPU executes one job completely before moving to next

Used in: Payroll systems, bank statement processing



Time-Sharing Operating System



Multiple users share CPU time simultaneously



CPU switches rapidly between users using time slices



Provides fast response and interactive usage



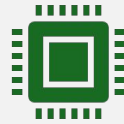
Used in: UNIX/Linux servers, university systems



Multiprogramming Operating System



Multiple programs kept in memory at the same time



CPU switches when one program waits for I/O



Improves CPU utilization



Used in: Early mainframe systems

Multitasking Operating System

Single user runs multiple applications at once

CPU switches between tasks very quickly

Gives illusion of parallel execution

Used in: PCs and laptops
(Windows, macOS)

Multiprocessing Operating System



USES MULTIPLE CPUS
OR CORES



TASKS ARE EXECUTED
IN PARALLEL



IMPROVES
PERFORMANCE AND
RELIABILITY



USED IN: SERVERS,
HIGH-PERFORMANCE
COMPUTERS



Confused?

Comparison Table

Feature	Multiprogramming	Multitasking	Time-Sharing	Multiprocessing
CPU	1	1	1	Multiple
Users	1	1	Multiple	Multiple
Execution	One at a time	Time-sliced	Time-sliced	Parallel
Main Goal	CPU utilization	Convenience	Response time	Performance

Real-Time Operating System (RTOS)



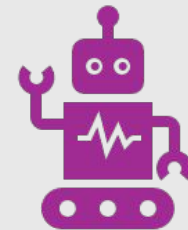
Guarantees response within a fixed time limit



Tasks are scheduled based on deadlines



Used where timing is critical



Used in: Aircraft systems, robots, medical devices

Embedded Operating System

Designed for specific
hardware with limited
resources

Performs dedicated functions
only

Often runs continuously
without user interaction

Used in: Washing machines,
routers, smart TVs



Distributed Operating System



Manages multiple computers as one system



Resources are shared across networked machines



Improves scalability and availability



Used in: Cloud computing, data centers

Network Operating System

Provides services to
computers over a network

Manages users, files, and
security

Each system works
independently

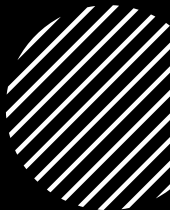
Used in: Office and
enterprise networks
(Windows Server)

Comparison

Feature	Network OS	Distributed OS
System View	Multiple separate systems	One unified system
User Awareness	User knows remote system	User sees single system
Resource Sharing	Explicit (manual)	Automatic (transparent)
Complexity	Lower	Higher



Mobile Operating System



Designed for handheld devices



Optimized for touch input and battery usage



Manages apps, sensors, and wireless communication



Used in: Smartphones and tablets
(Android, iOS)



Is one kernel design
suitable for all
operating systems?



Learning Outcomes



Identify the major types of kernels



Differentiate between monolithic, microkernel, and hybrid kernels

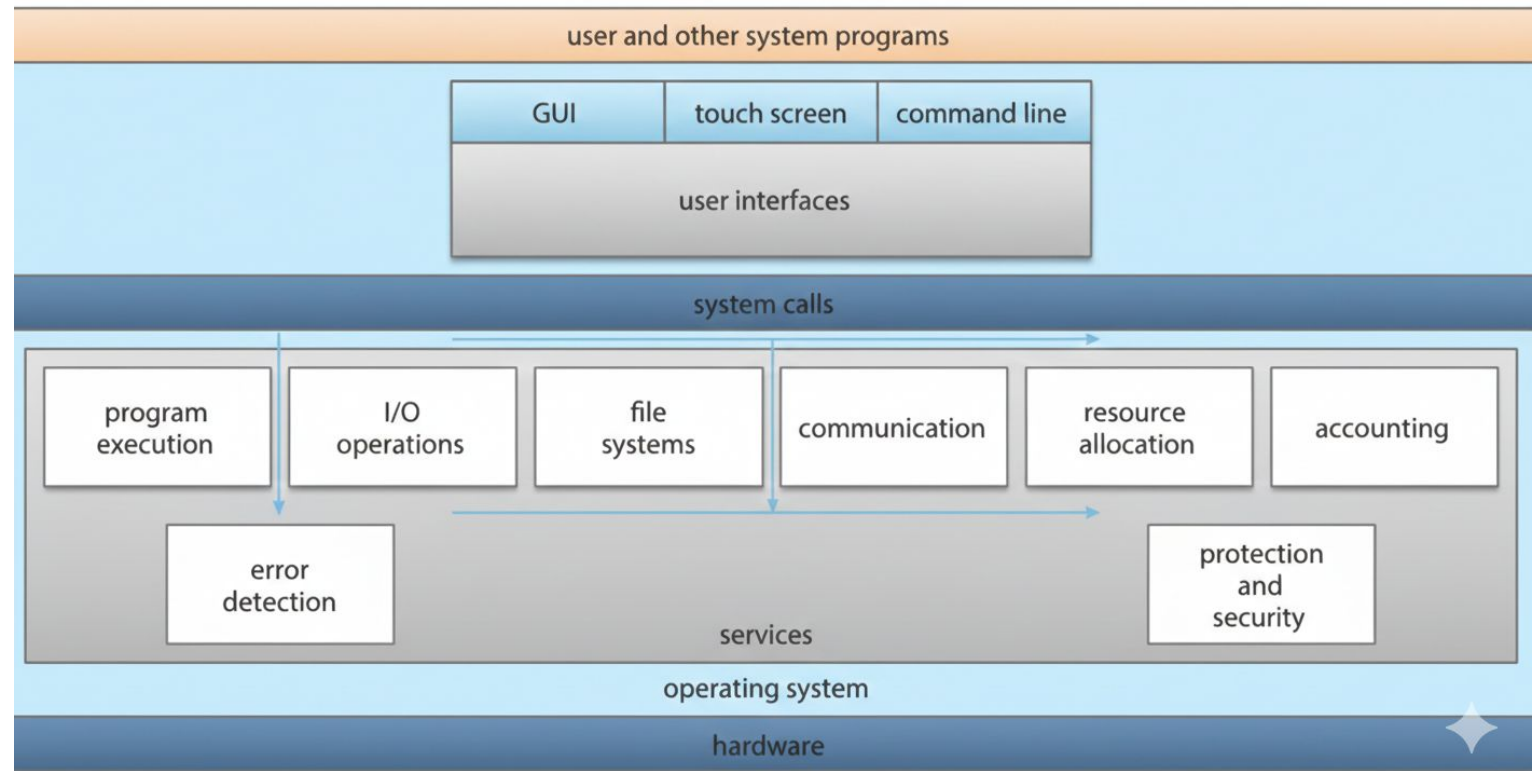


Explain the advantages and disadvantages of each kernel type



Give examples of operating systems using different kernel designs

General OS Structure





Why Different Kernel Types?



Different systems have different requirements



Performance, security, and flexibility vary



Kernel design affects OS efficiency and reliability



Main Types of Kernels



Simple Structure

Monolithic Kernel

Microkernel

Hybrid Kernel

Simple Structure

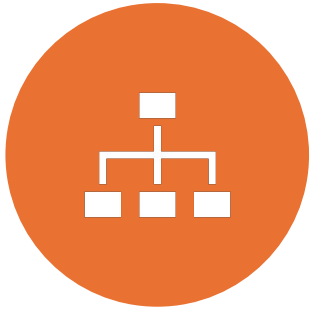
A simple structure means the OS is built as **one large program**

No clear separation between OS components

All functions are closely connected

Early operating systems used this approach

Key Characteristics



SINGLE-LEVEL
STRUCTURE



OS SERVICES AND
KERNEL FUNCTIONS
ARE MIXED



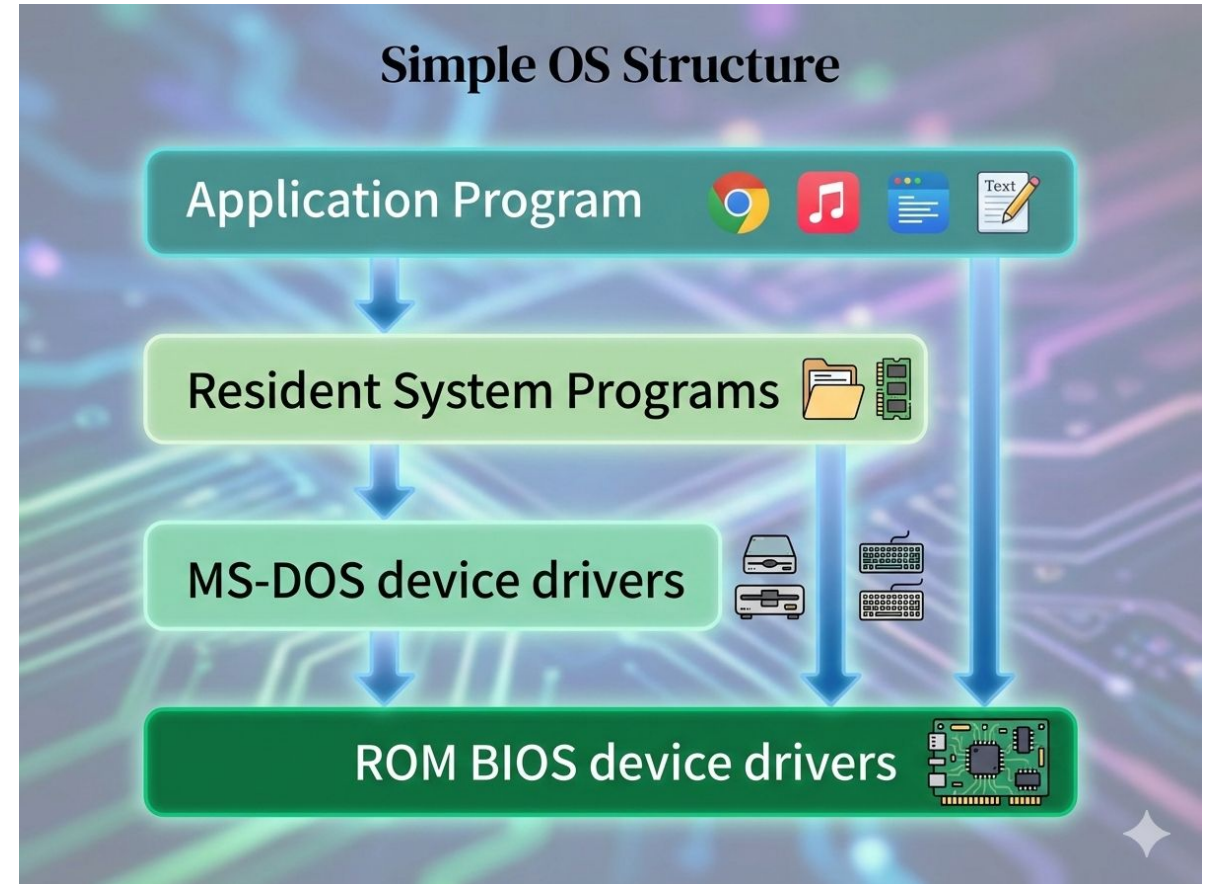
MINIMAL
ABSTRACTION



EASY TO DESIGN
INITIALLY

Example

- **MS-DOS (8088)**
- Simple OS structure
- No strict separation between user and kernel
- Applications could access hardware directly



Simple Structure

Advantages

- Simple to implement
- Low overhead
- Fast execution

Disadvantages

- Difficult to maintain
- Poor security
- A single error can crash the system
- Not scalable for modern systems



Monolithic Kernel



All core OS services run in **kernel space**

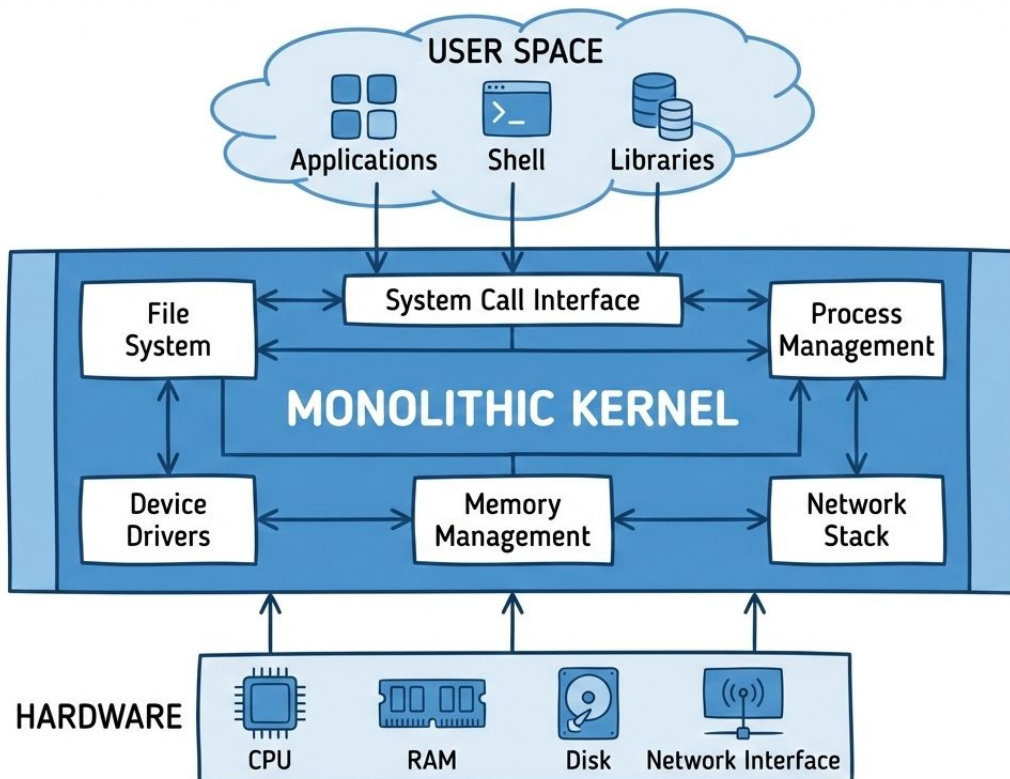
Services include:

- Process management
- Memory management
- Device drivers
- File system

Characteristics:

- Fast performance
- Large kernel size
- Less secure (a failure can crash the system)

Monolithic Kernel Working



All kernel components run as **one large program**



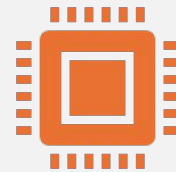
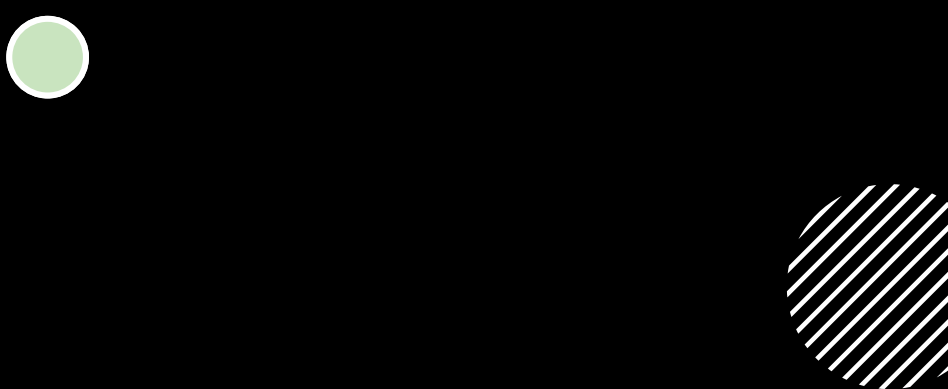
Components can directly communicate with each other



Runs in **kernel mode**



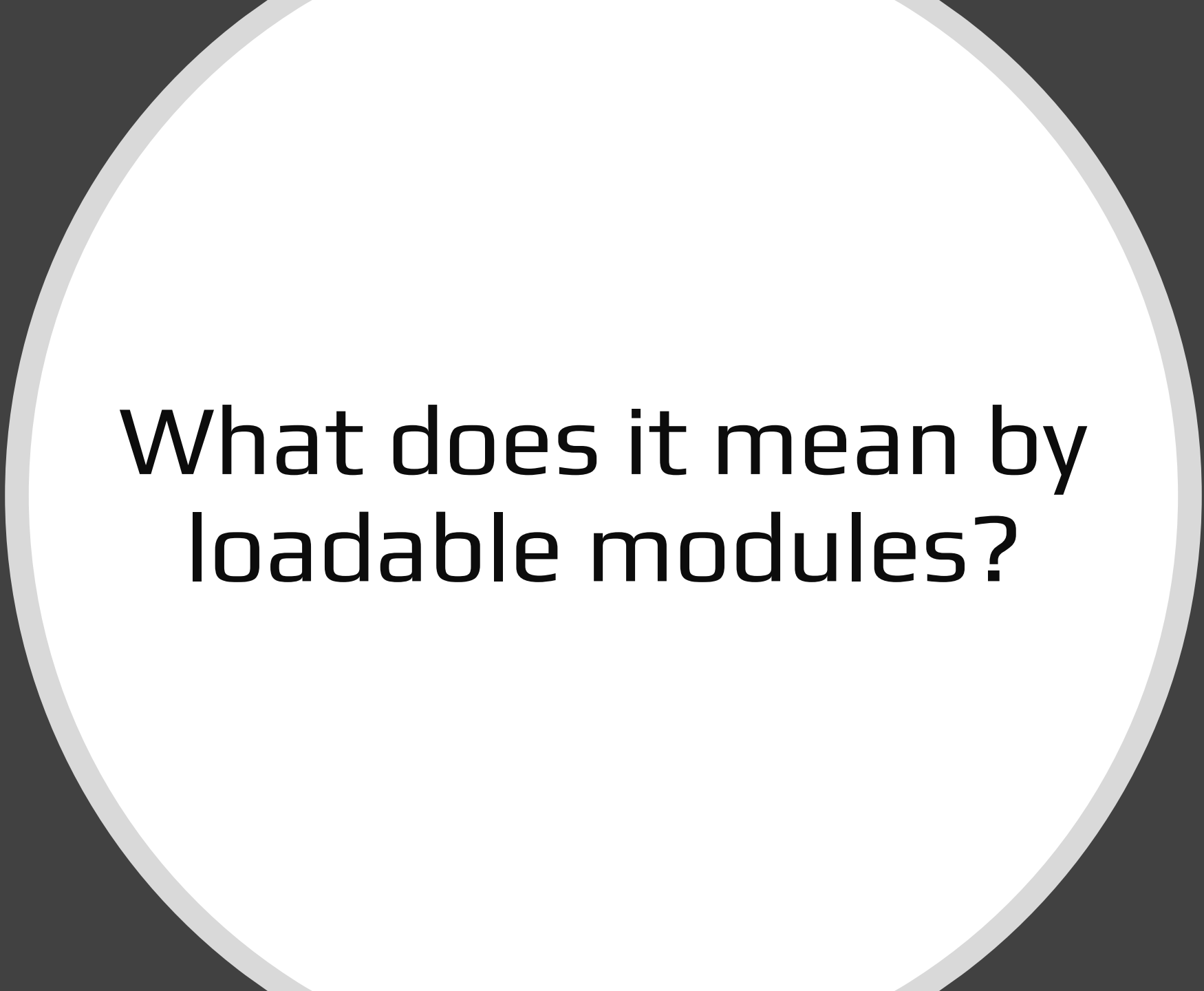
Monolithic Kernel – Examples



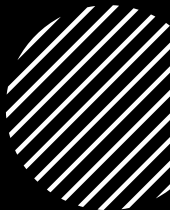
Linux Kernel, UNIX , BSD



Linux uses a monolithic
kernel with loadable
modules.



What does it mean by
loadable modules?



Modular Kernel



Core kernel is monolithic



Additional functionality is added as **loadable modules**



Modules can be loaded/unloaded at runtime



Example: Linux Kernel



Supports **loadable kernel modules (LKM)**

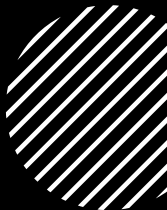
LKM

What you can add / remove

- **Hardware support**
(graphics card, network card, sound card)
- **File system support**
(USB drives, external disks, network storage)
- **Networking features**
(firewall, network sharing)
- **Extra kernel functionality**
(virtual machines, performance tools)

Why LKMs are useful

- No system reboot needed
- Kernel stays small and flexible
- Only load what you actually use



Microkernel

Only **essential services** run in kernel space

Other services run in **user space**

Kernel handles:

- Process scheduling
- Inter-process communication (IPC)
- Basic memory management

Characteristics:

- More secure
- More stable
- Slightly slower due to communication overhead

Microkernel – Examples

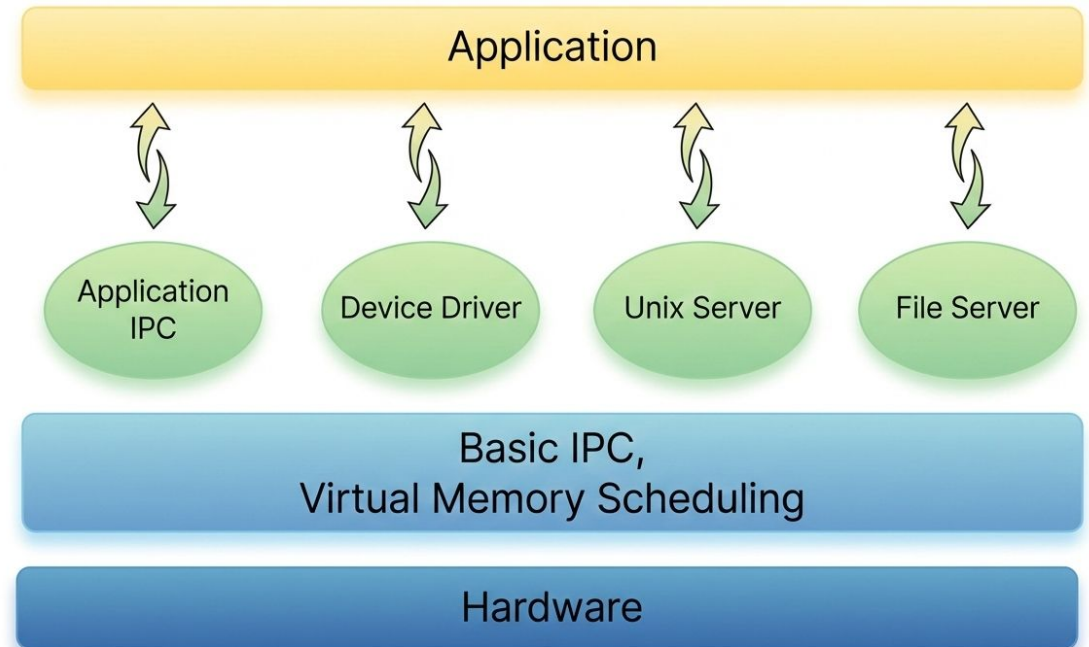


MINIX, QNX , Mach



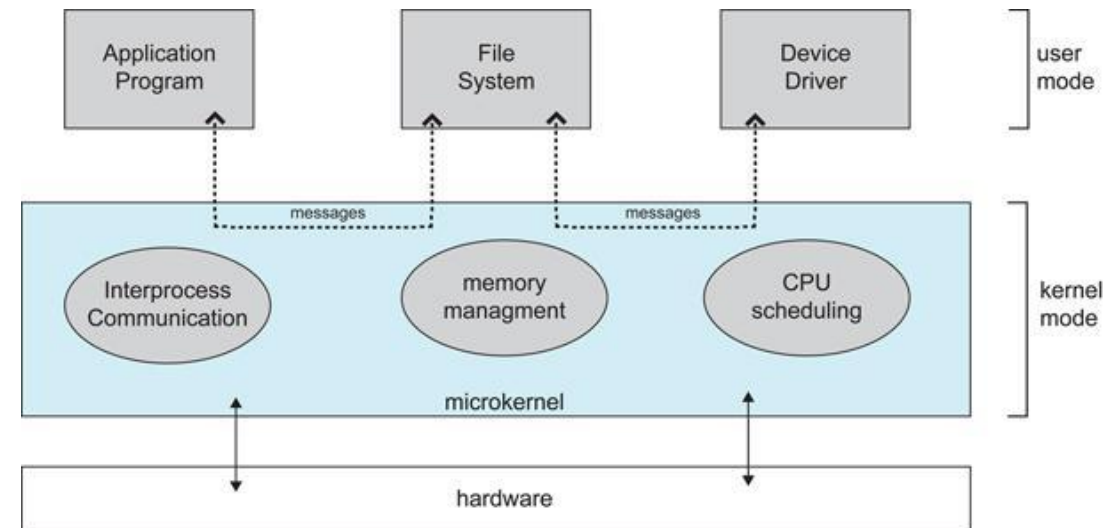
Microkernels focus on reliability and fault isolation.

MicroKernel Operating System

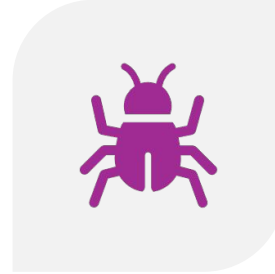


Microkernels Use Message Passing

- Kernel only handles:
 - **IPC (message passing)**
 - **Scheduling**
 - **Virtual memory**
- All OS services run in **user space**:
 - File system
 - Device drivers
 - UNIX/POSIX services
- Components communicate via **messages**, not shared kernel code



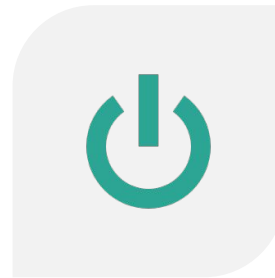
Why message passing?



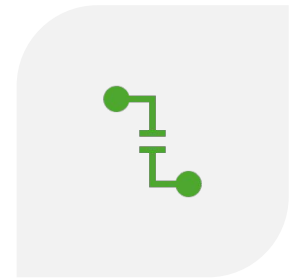
MINIMAL KERNEL → LESS
CODE, FEWER BUGS



FAULT ISOLATION →
CRASHED SERVICE ≠
SYSTEM CRASH



REPLACEABLE SERVICES
→ RESTART DRIVERS,
SWAP SERVERS



**MORE IPC/CONTEXT
SWITCHES**
(PERFORMANCE
TRADEOFF)

Monolithic VS Microkernel (Big Picture)

Monolithic Kernel: Faster

- Fewer context switches
- Direct function calls (no IPC)
- Better cache/TLB locality

Monolithic Kernel: Less secure / less fault-tolerant

- Drivers and services run in kernel space
- One buggy driver can crash the whole system
- Large trusted computing base (TCB)

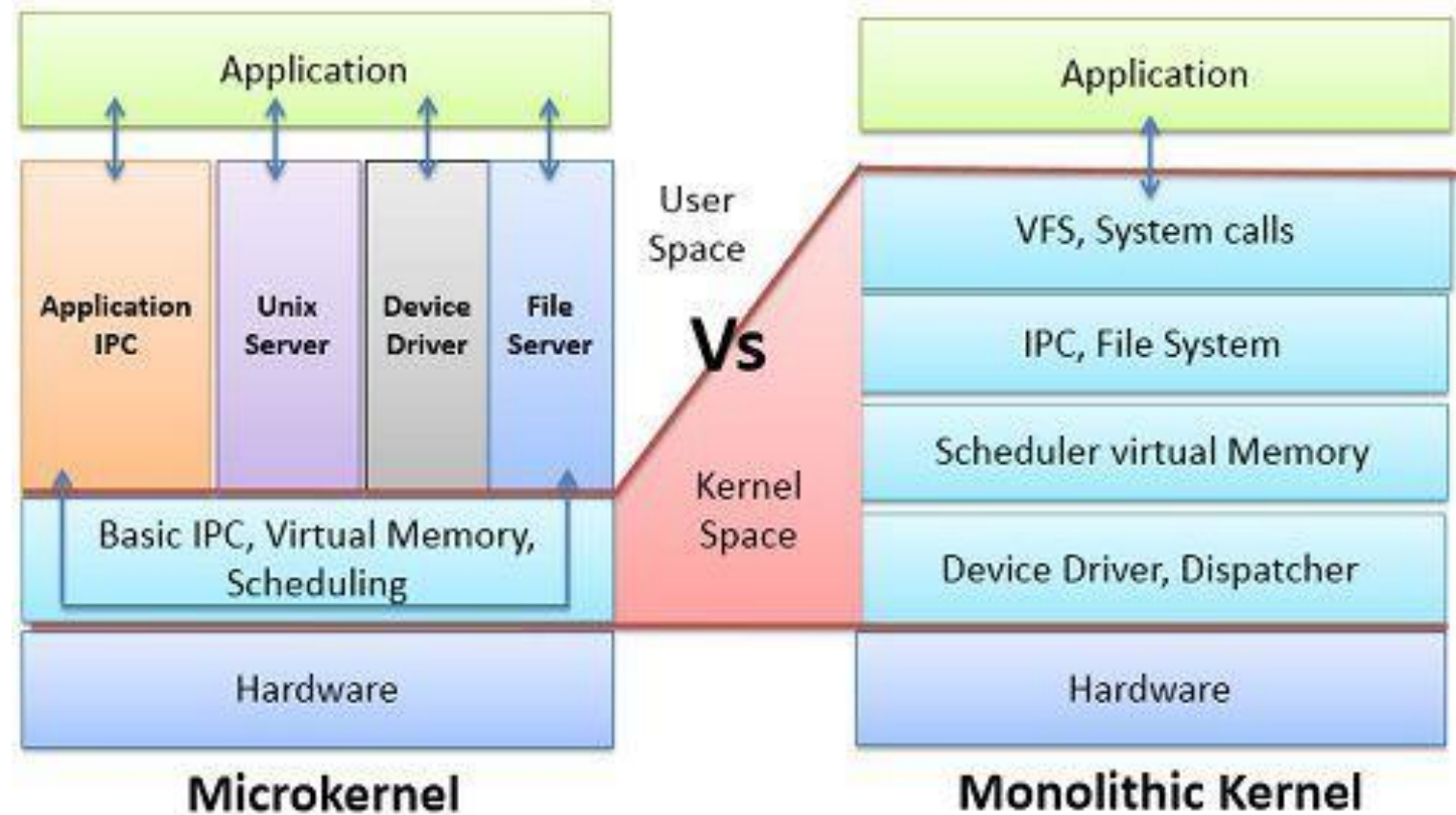
Microkernel: More secure & reliable

- Minimal kernel → small attack surface
- Services run in user space
- Strong isolation via message passing
- Crashed service ≠ system crash

Microkernel: Slower

- IPC overhead
- More context switches
- More complex data movement

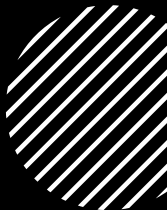
Monolithic vs Microkernel (Big Picture)



Any guesses which
kernel type windows
operating system
uses?



Hybrid Kernel



Combines features of **monolithic** and **microkernel**



Some services run in kernel space for performance



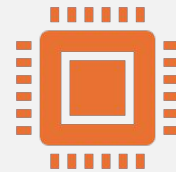
Others run in user space for stability



Goal: Balance performance and security



Hybrid Kernel – Examples



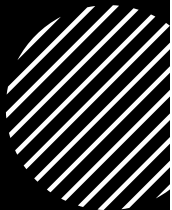
Windows NT, macOS (XNU kernel)



Hybrid kernels aim to get the best of both designs.



Layered Approach



The OS is divided into **multiple layers**



Each layer performs a **specific function**



Each layer uses services of the **layer below**



Lower layers are closer to **hardware**

Structure of Layered Approach



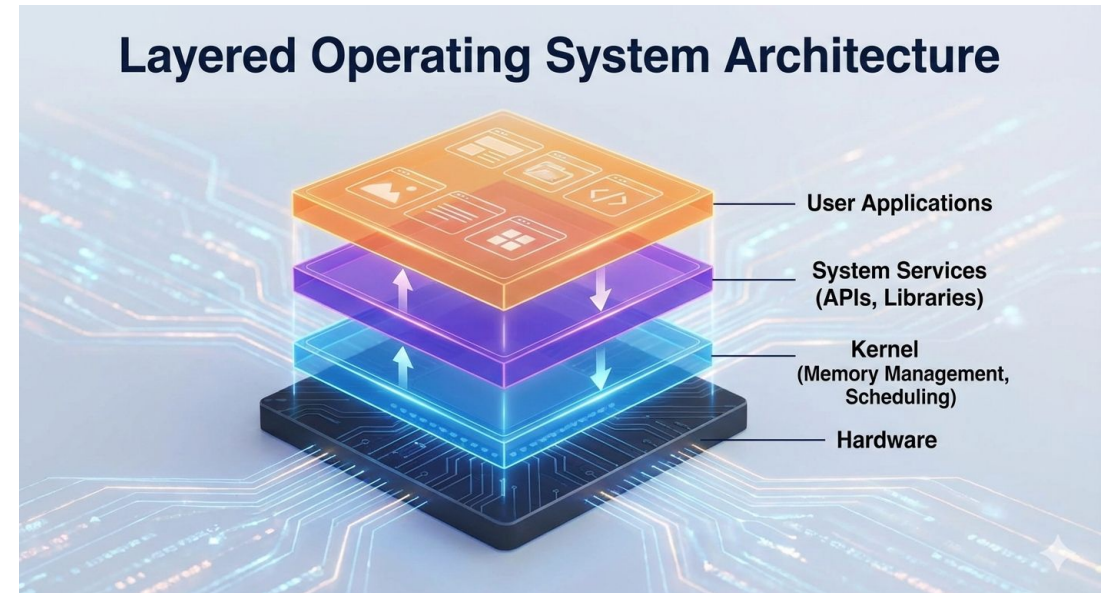
Bottom Layer:
Hardware



Middle Layers: Core
OS functions



Top Layer: User
interface and
applications



Layered Structure

Design approach, not a kernel type

OS is organized into **hierarchical layers**

Each layer:

- Uses services of the layer below
- Provides services to the layer above

Layered OS defines structure, not kernel architecture.

Can be applied to **monolithic**, **microkernel**, or **hybrid** systems

Layered Approach

Advantages

- Easy to **design and debug**
- Clear separation of responsibilities
- Better system organization
- Easier maintenance

Disadvantages

- Performance overhead due to multiple layers
- Difficult to define perfect layer boundaries
- Not suitable for high-performance systems

Do mobile phones use
a different kind of
kernel than
computers?

Mobile Operating System

An OS designed for **mobile devices**

Optimized for:

- Touch input
- Battery efficiency
- Sensors

Built on **existing kernel architectures**

Common Layers in Mobile OS

- **Applications**

User-installed apps (calls, messages, browser)

- **Application Framework**

APIs for UI, notifications, sensors, location

- **System Services / Runtime**

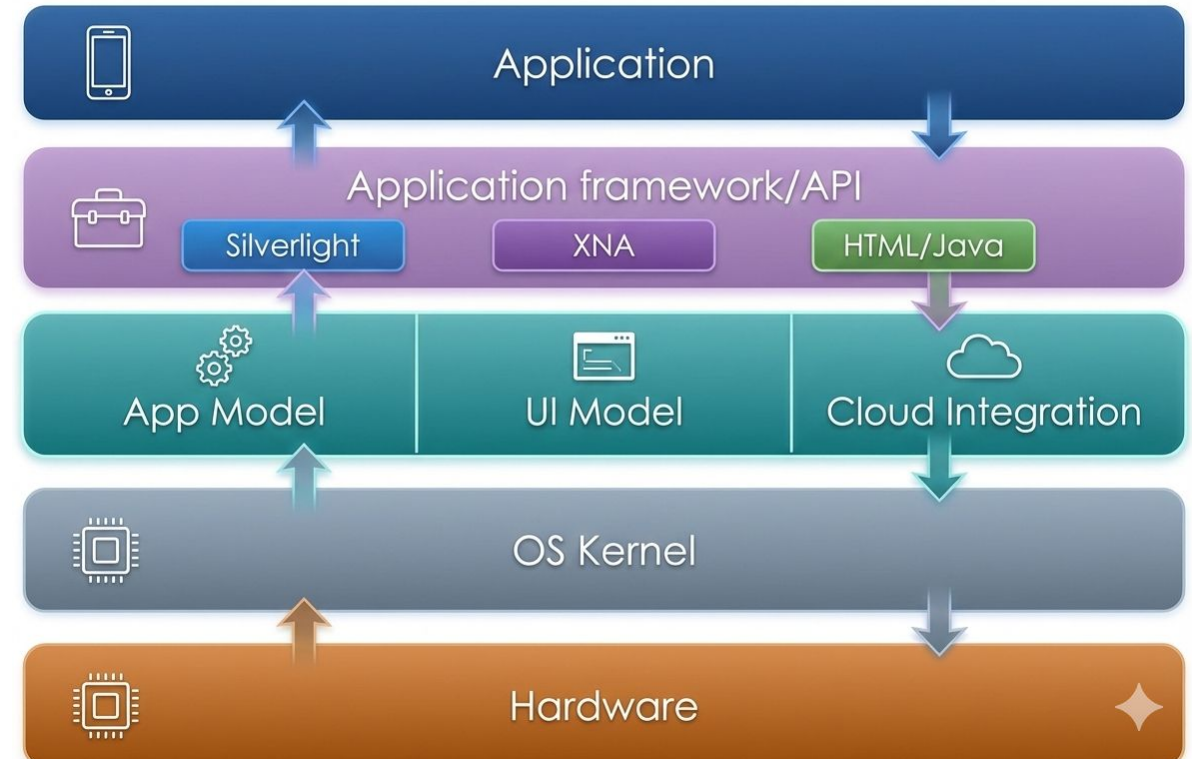
App execution, memory management, security

- **Kernel**

Process management, memory, device drivers

- **Hardware**

Mobile OS Architecture

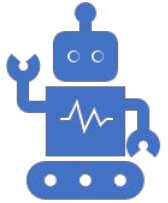


Mobile OS and Kernel Relationship

- Mobile OS do **not** introduce new **kernel types**
- They reuse:
 - Monolithic
 - Hybrid
 - Microkernel designs



Mobile OS Structures



Android

Uses **Linux kernel** (monolithic, modular)

Apps run in **separate sandboxes**

Open ecosystem

Hardware abstraction layer (HAL) separates hardware from OS



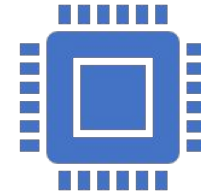
Apple (iOS)

Uses **XNU kernel** (hybrid)

Very **tight control** over apps and hardware

Strong app sandboxing

Hardware and software are tightly integrated



Windows Phone

Based on **Windows NT kernel**

Similar structure to desktop Windows

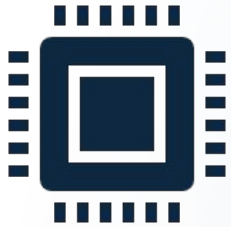
Strong isolation between apps

Discontinued, but architecturally important

Comparision

Aspect	Android	iOS	Windows Phone
Kernel	Linux	Hybrid (XNU)	Windows NT
Openness	Open	Closed	Closed
Hardware control	Manufacturer-driven	Apple-only	Microsoft
Customization	High	Low	Low

Summary



Modular design is part of monolithic kernels (Linux)



Layered approach is an OS design method



Mobile OS use existing kernel architectures

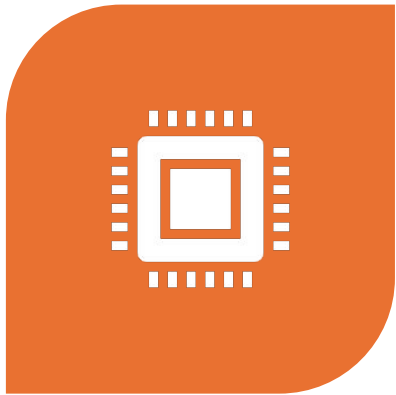


Comparison Table

Feature	Monolithic	Microkernel	Hybrid
Kernel size	Large	Small	Medium
Performance	High	Lower	High
Security	Lower	High	Medium
Stability	Lower	High	High
Complexity	Low	High	Medium



When to Use Which Kernel



MONOLITHIC:
GENERAL-PURPOSE OS
(LINUX)



MICROKERNEL: REAL-TIME
AND SAFETY-CRITICAL
SYSTEMS



HYBRID: DESKTOP AND
ENTERPRISE SYSTEMS

Summary



Kernel type affects OS performance and security



Monolithic kernels are fast but less secure



Microkernels are secure but slower



Hybrid kernels balance speed and stability